



Il Jai Shree Gurudev Il
Sri AdichunchanagiriShikshana Trust(R)
SJB Institute of Technology



(An Autonomous Institute under Visvesvaraya Technological University, Belagavi
Approved by AICTE, New Delhi, Recognized by UGC, Accredited by NAAC with 'A+' Grade
National Board of Accreditation)

No. 67, BGS Health & Education City, Dr. Vishnuvardhan Road,
Kengeri, Bengaluru-560060



DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

**A REPORT ON SPOTIFY
LISTENING DATA ANALYSIS AND
VISUALIZATION**

BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

**SUBMITTED BY:
K S VASANTH KARTHIK (1JB24CD022)**

Under The Guidance of:
Mrs. Jasma Bhavani,
Assistant Professor,
Department Of CSE(DS),
SJBIT

SJB INSTITUTE OF TECHNOLOGY

Bengaluru - 560060

Department Of Computer Science Engineering (Data Science)



CERTIFICATE

This is to certify that the work on entitled "**SPOTIFY LISTENING DATA ANALYSIS AND VISUALIZATION**" is a bonafide work carried out by **K S VASANTH KARTHIK** in partial fulfillment for the award of internal marks in Python Programming for Data Science during the odd semester of 2025- 2026. It is certified that all corrections/suggestions indicated for have been incorporated in the report.

Students Signature

Signature of Faculty

Signature of HOD

TABLE OF CONTENTS

INDEX	DESCRIPTION	PAGE NO
1.	ABSTRACT	4
2.	OBJECTIVES	5
3.	METHADOLOGY	6-7
4.	ALGORITHM USED AND PROGRAM	8-13
5.	OUTPUT AND APPLICATIONS	14-17
6.	CONCLUSION	18

ABSTRACT

With the rapid growth of digital music platforms, large volumes of user listening data are generated daily. This project focuses on analyzing and visualizing **Spotify Extended Streaming History** data using **Python**. The system processes **JSON-based streaming history files** to extract meaningful insights such as **daily listening trends, average listening behavior across weekdays, top artists, top tracks, and long-term listening patterns.**

The project makes use of Python libraries such as **Pandas, NumPy, Matplotlib, and Seaborn** for data preprocessing and visualization. By converting raw timestamped data into structured time-series formats, the project demonstrates how real-world user data can be transformed into actionable insights. This work highlights practical applications of data analysis, aggregation, and visualization techniques in the field of data science.

OBJECTIVES

The main objectives of this project are:

1. To analyze **Spotify listening history** data using **Python**.
2. To preprocess and clean **JSON-based streaming data**.
3. To **visualize** listening behavior over time using **bar charts** and **line graphs**.
4. To analyze **average listening habits** across different **days of the week**.
5. To identify **top artists** and **tracks** based on listening **frequency** and **duration**.
6. To demonstrate the use of Python libraries for **real-world data analysis tasks**.

METHODOLOGY

1. Data Collection

- Spotify Extended Streaming History files are collected in JSON format. These files contain details such as artist name, track name, timestamp, and duration of each stream.

2. Data Preprocessing

- JSON files are read and converted into Pandas DataFrames.
- Timestamp values are converted into datetime format.
- Listening duration is converted from milliseconds to hours and minutes.
- Missing dates in the time series are handled using frequency normalization.

3. Data Aggregation

- Daily listening time is calculated by grouping data by date.
- Average listening behavior is computed for each weekday.

- Artists and tracks are ranked based on total streams and total listening time.

4. Data Visualization

- Bar charts are used to represent daily listening trends.
- Pie charts and bar graphs are used to analyze weekday-wise listening patterns.
- Dual-axis bar charts are used to compare number of streams and listening duration.
- Line charts are used to visualize listening history of top artists over time.

5. User Interaction

- The user can select different date ranges such as full history, specific year, month, or custom range for analysis.

ALGORITHM USED

Spotify Listening Analysis Algorithm

1. Start the program.
2. Load Spotify streaming history JSON files.
3. Load Spotify streaming history JSON files.
4. Convert raw data into a unified DataFrame.
5. Group data by date to calculate daily listening time.
6. Normalize the data to include days with zero activity.
7. Compute weekday-wise averages for listening time and stream count.
8. Identify top artists and tracks based on streams and duration.
9. Generate visualizations using Matplotlib.
10. Display results and plots to the user.
11. End.

PROGRAM

```
# Import required libraries for data analysis and visualization
import pandas as pd
import os
import json
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn
import numpy as np

# Set the seaborn theme for plot styling
seaborn.set_theme(style="white", palette="pastel")

# Convert milliseconds to hours
def ms2hr(ms_val):
    return ms_val / (1000 * 60 * 60)

# Calculate total listening time per day
def load_over_time(df):
    df['endTime'] = pd.to_datetime(df['endTime'])
    df['endTime'] = pd.to_datetime(df['endTime'].dt.strftime("%Y-%m-%d"))

    df_time = df[['endTime', 'msPlayed']]

    df_time_sum = df_time.groupby(['endTime'], as_index=False).agg({'msPlayed': 'sum'})
    df_time_sum['hrPlayed'] = df_time_sum['msPlayed'] / (1000 * 60 * 60)

    return df_time_sum

# Create a bar chart with date formatting on x-axis
def plot_df(df, x, y, title=None, y_label=None):
    fig, ax = plt.subplots(1, 1)
    ax.bar(x, y, data=df)

    if title is not None:
        ax.set_title(title)
    if y_label is not None:
        ax.set_ylabel(y_label)

    fmt_month = mdates.MonthLocator(interval=1)
    ax.xaxis.set_major_locator(fmt_month)

    fmt_day = mdates.DayLocator()
    ax.xaxis.set_minor_locator(fmt_day)

    ax.grid(True)

    fig.autofmt_xdate()

# Analyze and visualize average streaming patterns by day of week
def avg_day_load(df):
    df['endTime'] = pd.to_datetime(df['endTime'])
    df['date'] = pd.to_datetime(df['endTime'].dt.strftime('%Y-%m-%d'))
    df = df.groupby(['date'], as_index=True).agg({'endTime': 'count', 'msPlayed': 'sum'})
    df = df.asfreq('D', fill_value=0) # Fill missing dates with zeros
    df.reset_index(level=0, inplace=True)
    days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    day_types = pd.CategoricalDtype(categories=days, ordered=True)
    df['weekday'] = df['date'].dt.day_name()
    df['weekday2'] = df['weekday'].astype(day_types)
    df['weekday'] = df['weekday'].astype(day_types)

    # Aggregate data by weekday
    df_week_sum = df.groupby('weekday', as_index=True) \
        .agg({'weekday2': 'count', 'endTime': 'sum', 'msPlayed': 'sum'}) \
```

```

        .rename(columns={'endTime':'noStreams','weekday2':'dayCount'})

# Calculate average statistics per weekday
df_week_sum['hrPlayed'] = df_week_sum['msPlayed'] / (1000 * 60 * 60)
df_week_sum['hrPlayedAvg'] = df_week_sum['hrPlayed'] / df_week_sum['dayCount']
df_week_sum['noStreamsAvg'] = df_week_sum['noStreams'] / df_week_sum['dayCount']
df_week_sum['lenStreamsAvgMin'] = df_week_sum['msPlayed'] / df_week_sum['noStreams'] / (1000 * 60)

df_week_sum = df_week_sum.drop(columns=['msPlayed','hrPlayed','noStreams','dayCount'])

# Create 6 subplots: 3 pie charts and 3 bar charts
fig = plt.figure()
ax = fig.add_subplot(231)
ax2 = fig.add_subplot(232)
ax3 = fig.add_subplot(233)

# Pie charts for distribution across weekdays
ax.axis('equal')
ax.pie(df_week_sum['hrPlayedAvg'],labels=days,autopct='%1.2f%%')
ax.set_title('Avg Stream Time per Day')

ax2.axis('equal')
ax2.pie(df_week_sum['noStreamsAvg'],labels=days,autopct='%1.2f%%')
ax2.set_title('Avg No. of Streams per Day')

ax3.axis('equal')
ax3.pie(df_week_sum['lenStreamsAvgMin'],labels=days,autopct='%1.2f%%')
ax3.set_title('Avg Stream Length per Day')

# Bar charts for comparison
axb = fig.add_subplot(234)
axb.bar(days,df_week_sum['hrPlayedAvg'])
axb.set_ylabel('Hours Played')

axb2 = fig.add_subplot(235)
axb2.bar(days,df_week_sum['noStreamsAvg'])
axb2.set_ylabel('No. of Streams')

axb3 = fig.add_subplot(236)
axb3.bar(days,df_week_sum['lenStreamsAvgMin'])
axb3.set_ylabel('Avg Stream Length (Min)')
fig.autofmt_xdate()

return df_week_sum

# Get top artists by number of streams with dual-axis visualization
def top_artists(df, top=10, date_desc=''):
    # Group by artist and calculate total streams and play time
    df_top=df.groupby('artistName',as_index=False) \
        .agg({'endTime':'count','msPlayed':'sum'}) \
        .rename(columns={'endTime':'noStreams','msPlayed':'streamTimeMs'})
    df_top['streamTimeHr'] = ms2hr(df_top['streamTimeMs'])
    df_top = df_top.sort_values(by=['noStreams'], ascending=False)
    df_top = df_top.head(top)

    # Create dual-axis bar chart
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax2 = ax.twinx()
    ax2.grid(False)

    width = 0.27
    ind = np.arange(len(df_top))

    bar1 = ax.bar(ind, df_top['noStreams'], width, color = 'r', label='No. of Streams')
    bar2 = ax2.bar(ind + width, df_top['streamTimeHr'], width, color = 'b', label='Stream Time (Hr)')
    fig.legend(loc='upper right')

    ax.set_xticks(ind + width)

    ax.set_xticklabels(df_top['artistName'])
    ax.set_title(f'Top {top} Artists by No. of Streams {date_desc}')
    ax.set_ylabel('No. of Streams')

    ax2.set_ylabel('Stream Time (Hr)')
    fig.autofmt_xdate()

    return df_top

```

```

# Get top tracks by number of streams with dual-axis visualization
def top_tracks(df, top=10, date_desc=''):
    # Group by artist and track, calculate stats
    df_top=df.groupby(['artistName','trackName'],as_index=False) \
        .agg({'endTime':'count','msPlayed':'sum'}) \
        .rename(columns={'endTime':'noStreams','msPlayed':'streamTimeMs'})
    df_top['streamTimeHr'] = ms2hr(df_top['streamTimeMs'])
    df_top = df_top.sort_values(by=['noStreams'],ascending=False)
    df_top = df_top.head(top)
    df_top['fullName'] = df_top['artistName'] + " - " + df_top['trackName']

    # Create dual-axis bar chart
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax2 = ax.twinx()
    ax2.grid(False)

    width = 0.27
    ind = np.arange(len(df_top))

    bar1 = ax.bar(ind,df_top['noStreams'], width, color = 'r', label='No. of Streams')
    bar2 = ax2.bar(ind + width, df_top['streamTimeHr'], width, color = 'b', label='Stream Time (Hr)')
    fig.legend(loc='upper right')

    ax.set_xticks(ind + width)
    ax.set_xticklabels(df_top['fullName'])
    ax.set_title(f'Top {top} Tracks by No. of Streams {date_desc}')
    ax.set_ylabel('No. of Streams')

    ax2.set_ylabel('Stream Time (Hr)')
    fig.autofmt_xdate()

    return df_top

# Plot streaming history over time for top artists
def top_artists_history(df, top_artists_df=10, date_desc=''):
    # Filter to only include top artists
    df_top = df[df['artistName'].isin(top_artists_df['artistName'])]
    df_top['endTime'] = pd.to_datetime(df_top['endTime'])

    df_top['date'] = pd.to_datetime(df_top['endTime'].dt.strftime('%Y-%m-%d'))
    df_top = df_top.groupby(['artistName','date'],as_index=False) \
        .agg({'endTime':'count'}) \
        .rename(columns={'endTime':'noStreams'})

    # Create line chart for each artist
    fig = plt.figure()
    ax = fig.add_subplot(111)

    for artist in top_artists_df['artistName']:
        df_tmp = df_top[df_top['artistName'] == artist]
        ax.plot(df_tmp['date'],df_tmp['noStreams'],'-o',label=artist)
    ax.legend()
    ax.set_title(f'Top {len(top_artists_df)} Artists Streaming History {date_desc}')
    ax.set_ylabel("No. of Streams")

    return df_top

# Get artists listened to on the most unique days
def top_artists_most_days(df, top=10, date_desc=''):
    df = df.copy()
    df['endTime'] = pd.to_datetime(df['endTime'])
    df['date'] = pd.to_datetime(df['endTime'].dt.strftime('%Y-%m-%d'))
    df = df[['artistName', 'date']].drop_duplicates() # Remove duplicate artist-date pairs

    # Count unique days per artist
    df = df.groupby('artistName',as_index=False) \
        .agg({'date':'count'}) \
        .rename(columns={'date':'noDays'})
    df.sort_values(by=['noDays'],ascending=False,inplace=True)

    df = df.head(top)

    # Create bar chart
    width = 0.7
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.set_title(f'Top {top} Artists by Most Days Listened {date_desc}')
    ax.set_ylabel('No. of Days')

```

```

bar1 = ax.bar(df['artistName'],df['noDays'],width,color='r')
fig.autofmt_xdate()

return df

# Load and combine multiple JSON streaming history files
def file2df(stream_file_list):
    dfs = []
    for f_name in stream_file_list:
        with open(f_name, encoding="utf-8") as f:
            df_from_json = pd.json_normalize(json.loads(f.read()))
            dfs.append(df_from_json)

    df = pd.concat(dfs, sort=False)
    return df

# Interactive menu for user to select date range for analysis
def choose_date_range(df):
    while True:
        print("Choose Date Range for Analysis:")
        print("1. Full History")
        print("2. Choose a Date Range")
        print("3. Choose a Specific Year")
        print("4. Choose a Specific Month")
        print("5. Exit")
        choice = input("Select an option (1-5): ")
        df['endTime'] = pd.to_datetime(df['endTime'])

        # Option 1: Full history
        if choice == '1':
            return 1,df,f'Full History'

        # Option 2: Custom date range
        elif choice == '2':
            start_date = input("Enter start date (YYYY-MM-DD): ")
            if start_date not in df['endTime'].dt.strftime('%Y-%m-%d').values:
                print("Error: Start date not in data range.")
                continue
            end_date = input("Enter end date (YYYY-MM-DD): ")
            if end_date not in df['endTime'].dt.strftime('%Y-%m-%d').values:
                print("Error: End date not in data range.")
                continue
            if start_date > end_date:
                print("Error: Start date must be before end date.")
                continue
            return 1,df[(df['endTime'] >= start_date) & (df['endTime'] <= end_date)],f'from {start_date} to {end_date}'

        # Option 3: Specific year
        elif choice == '3':
            year = input("Enter year (YYYY): ")
            if year not in df['endTime'].dt.strftime('%Y').values:
                print("Error: Year not in data range.")
                continue
            return 1,df[df['endTime'].dt.year == int(year)],f'for the year {year}'

        # Option 4: Specific month
        elif choice == '4':
            year = input("Enter year (YYYY): ")
            if year not in df['endTime'].dt.strftime('%Y').values:
                print("Error: Year not in data range.")
                continue
            month = input("Enter month (1-12): ")
            if month.zfill(2) not in df['endTime'].dt.strftime('%m').values[df['endTime'].dt.strftime('%Y') == year]:
                print("Error: Month not in data range for the specified year.")
                continue
            return 1,df[(df['endTime'].dt.year == int(year)) & (df['endTime'].dt.month == int(month))],f'for {year}-{month.zfill(2)}'

        # Option 5: Exit
        elif choice == '5':
            print("Exiting...")
            return 0,df,''

# Main function to orchestrate the analysis
def main(stream_file_list):
    while True:
        # Load data from JSON files
        df = file2df(stream_file_list)
        # Rename columns for easier access

```

```

df.rename(columns={'ts':'endTime','master_metadata_album_artist_name':'artistName',
                  'master_metadata_track_name': 'trackName','ms_played':'msPlayed'},
          inplace=True)

# Get user's selected date range
end, df_date, date_desc = choose_date_range(df)

if end == 0:
    exit(0)
else:
    # Generate all visualizations and analyses
    df_listen_time = load_over_time(df_date)
    plot_df(df_listen_time, 'endTime', 'hrPlayed',
            title=f"Listening time to Spotify streams per day: {date_desc}",
            y_label='Hours [h]')
    df_avg_day = avg_day_load(df_date)
    df_top_artists = top_artists(df_date,10, date_desc)
    df_top_tracks = top_tracks(df_date,10, date_desc)
    df_top_artists_history = top_artists_history(df_date,df_top_artists.head(5), date_desc)
    df_top_artists_most_days = top_artists_most_days(df_date,10, date_desc)
    plt.show()

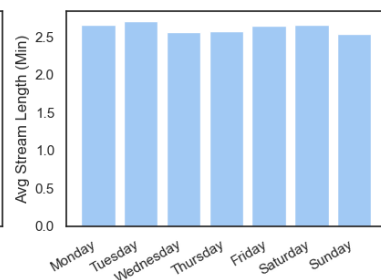
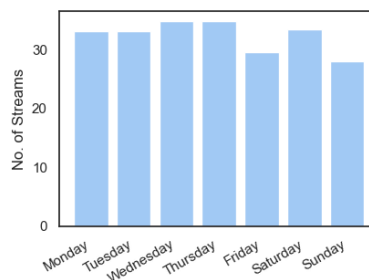
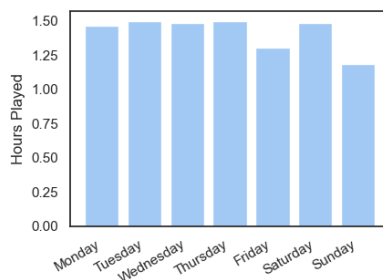
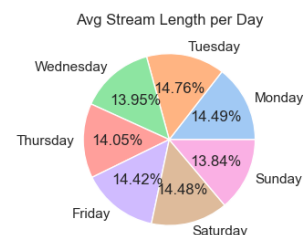
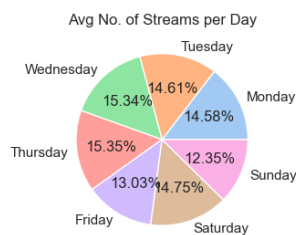
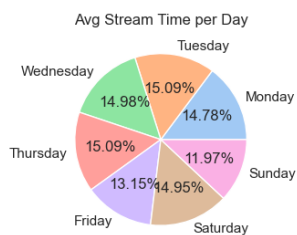
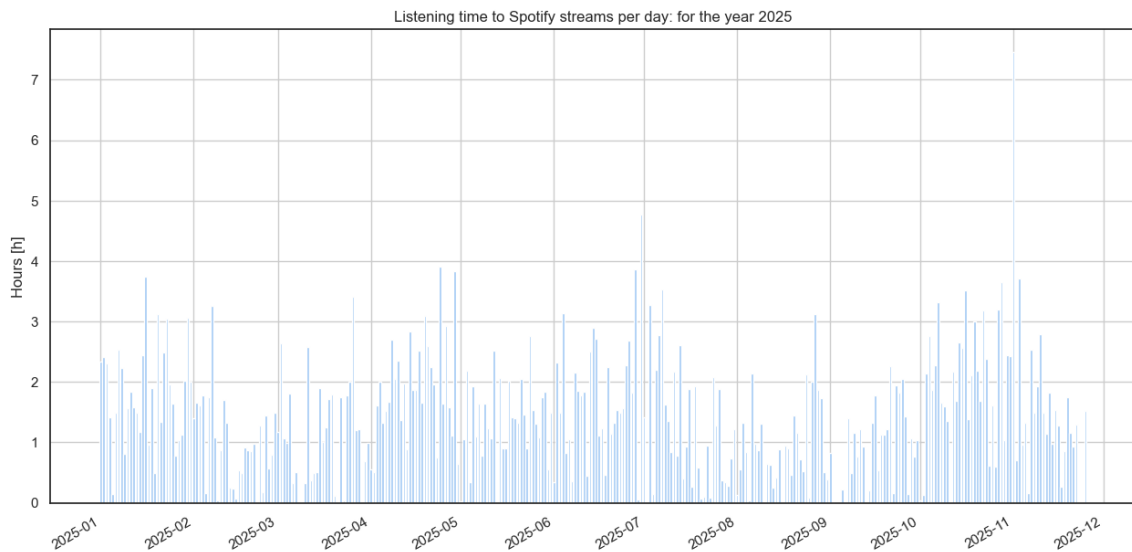
# Entry point: Load JSON files and run analysis
if __name__ == "__main__":
    # Get directory of this script
    base_dir = os.path.dirname(os.path.abspath(__file__))
    base_dir = os.path.join(base_dir, 'Spotify Extended Streaming History')
    # Find all streaming history JSON files
    all_files = [os.path.join(base_dir, f) for f in os.listdir(base_dir)
                 if f.startswith("Streaming_History_") and f.lower().endswith('.json')]

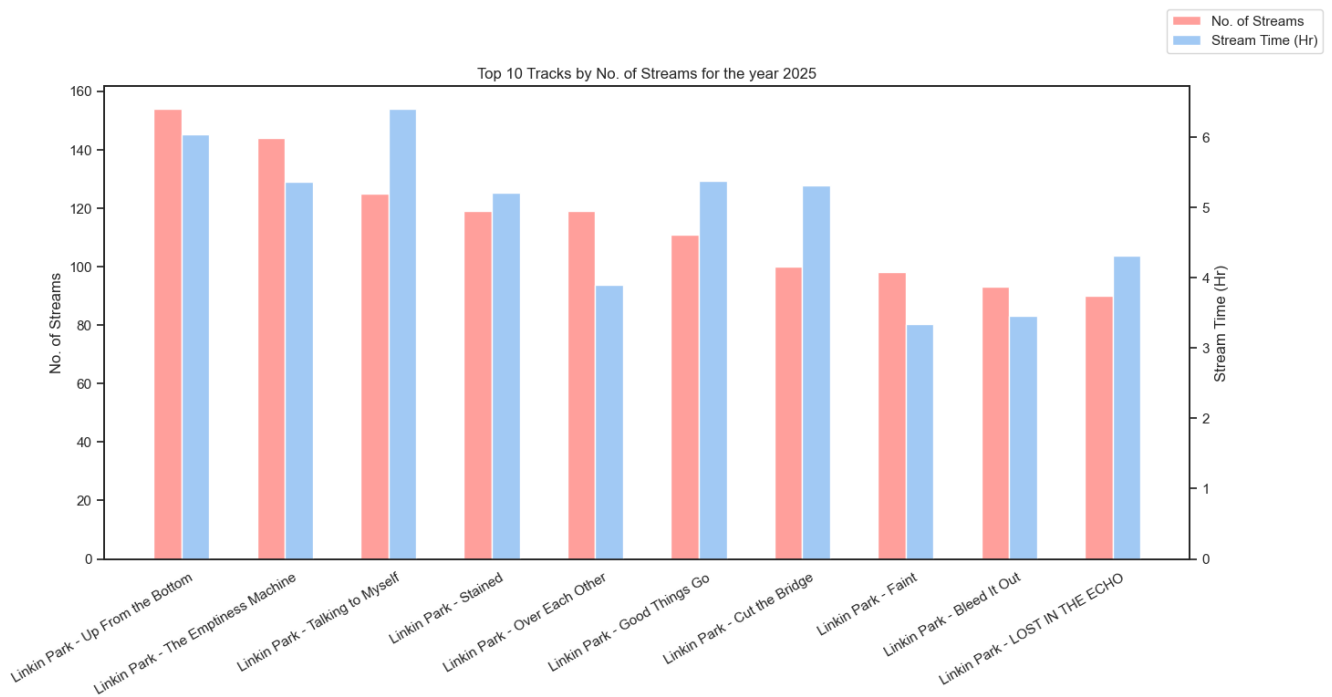
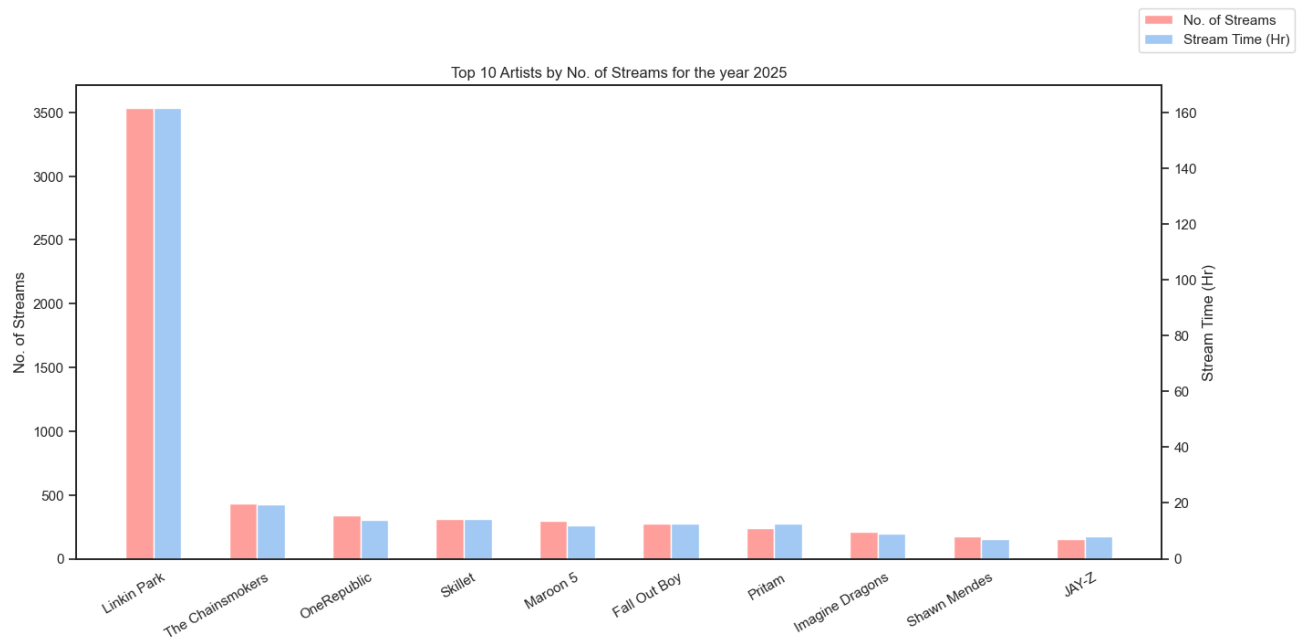
    if all_files:
        main(all_files)
    else:
        print(f"No JSON file found or no file with the naming convention Streaming_History_Audio.json found")

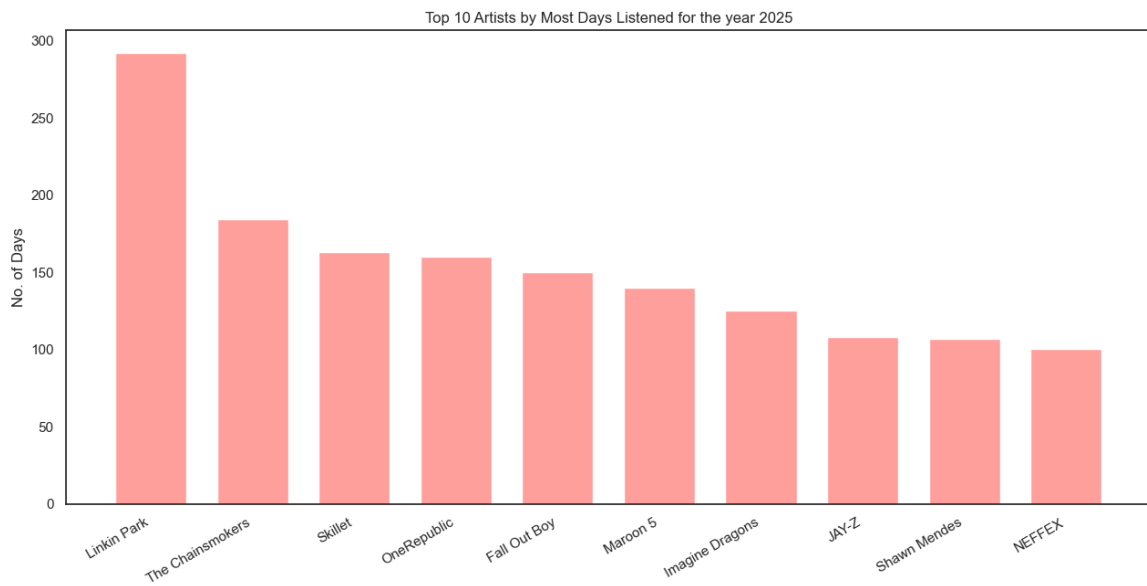
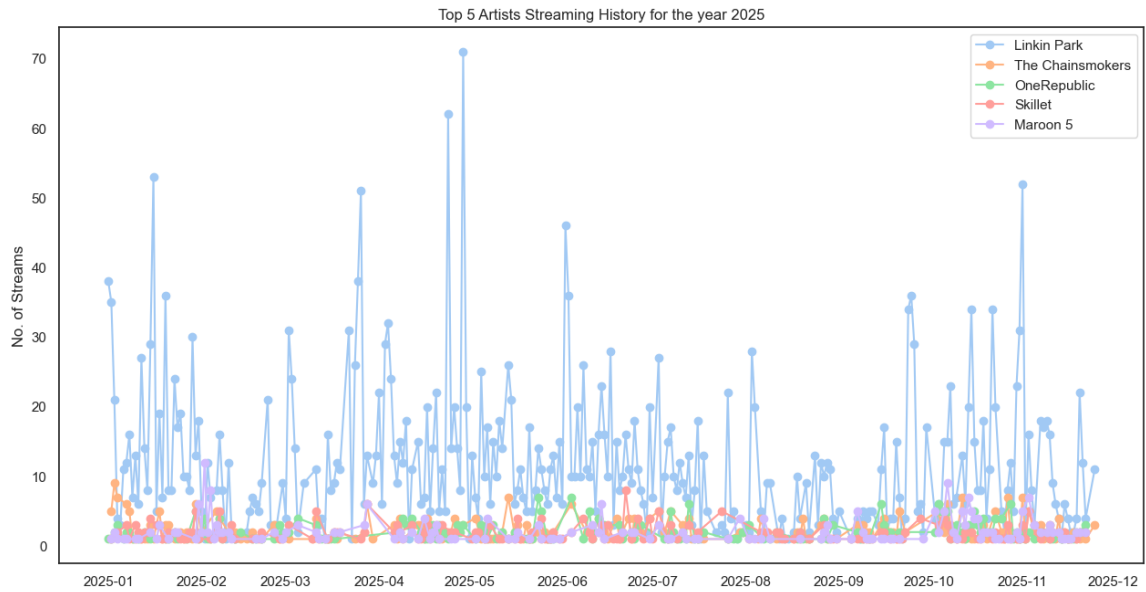
```

OUTPUT

```
PS D:\Projects\College\3rd Sem\Python\Spotify Activity Analyzer> & C:\Python313\python.exe "d:/Projects/College/3rd Sem/Python/Spotify Activity Analyzer/spotify_activity_analyzer.py"
Choose Date Range for Analysis:
1. Full History
2. Choose a Date Range
3. Choose a Specific Year
4. Choose a Specific Month
5. Exit
Select an option (1-5): 3
Enter year (YYYY): 2025
```







```

Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Choose Date Range for Analysis:
1. Full History
2. Choose a Date Range
3. Choose a Specific Year
4. Choose a Specific Month
5. Exit
Select an option (1-5): 5
Exiting...
PS D:\Projects\College\3rd Sem\Python\Spotify Activity Analyzer>

```


APPLICATIONS

The Spotify Listening Data Analysis project has several applications, including:

1. Personal Listening Insights Users can understand their music consumption habits and preferences.
2. Recommendation Analysis Helps in understanding trends that can support music recommendation systems.
3. Behavioral Analysis Useful for studying user behavior patterns in digital media platforms.
4. Data Science Learning Demonstrates practical implementation of data preprocessing, aggregation, and visualization techniques.
5. Time-Series Analysis Serves as an example of real-world time-series data analysis.
6. Academic Mini Project Suitable as a data science mini project for undergraduate studies.

ALGORITHM USED

The **Spotify Listening Data Analysis and Visualization** project successfully demonstrates how raw streaming data can be transformed into meaningful insights using **Python**. Through effective **preprocessing, aggregation, and visualization techniques**, the project highlights listening **trends, artist preferences, and behavioral patterns**. This project strengthens understanding of data analysis concepts and provides a **strong foundation for advanced analytics applications** such as **recommendation systems** and **predictive modeling**.

Future enhancements may include integration with **machine learning models, interactive dashboards, or cloud-based deployment** for large-scale data analysis.